

Nr. 1

1. erstellen Sie einen deterministischen PDA, der die Sprache $L = \{ w \in \{ a, b, c \}^* \mid w \text{ hat doppelt so viele a's wie c's} \}$ akzeptiert.

(PDA) = (Q (Zustände), Σ (Eingabesymbole), Γ (Stackalphabet), δ (Übergangsfunktionen), q_0 (Startzustand), $\perp$ (Leerer Stack), F (akzeptierende Zustände/Endzustände))

(PDA) = ($Q = \{ q_0, q_{\text{c}}, \text{accept} \}$, $\Sigma = \{ a, b, c \}$, $\Gamma = \{ \perp, P, M \}$, δ (Übergangsfunktionen), q_0 , $\perp$, $F = \{ \text{accept} \}$)

—

Noah: geht das auch?

$F = \emptyset$ und ein gültiger Endpunkt ist irgendein Zustand z.B. q mit leerem Stack.

PDA definiert durch leeren Stack als Akzeptanzkriterium.

also: Akzeptiert wenn Keller leer

$\delta(q, a, C) = \{(q, \epsilon)\}$

$\delta(q, a, \perp) = \{(q, A\perp)\}$

$\delta(q, a, A) = \{(q, AA)\}$

$\delta(q, b, C) = \{(q, C)\}$

$\delta(q, b, \perp) = \{(q, \perp)\}$

$\delta(q, b, A) = \{(q, A)\}$

$\delta(q, c, C) = \{(q, CCC)\}$

$\delta(q, c, \perp) = \{(q, CC\perp)\}$

$\delta(q, c, A) = \{(q, \epsilon)\}$

—

$\delta(q_0, a, \perp) = (q_0, M\perp)$

$\delta(q_0, b, \perp) = (q_0, \perp)$

$\delta(q_0, c, \perp) = (q_0, P P \perp)$

$\delta(q_0, a, M) = (q_0, M M)$

$\delta(q_0, b, M) = (q_0, M)$

$\delta(q_0, c, M) = (q_{\text{c}}, \epsilon)$

$\delta(q_0, a, P) = (q_0, \epsilon)$

$\delta(q_0, b, P) = (q_0, P)$

$\delta(q_0, c, P) = (q_0, P P P)$

$\delta(q_{\text{c}}, \epsilon, \perp) = (q_0, P \perp)$

$\delta(q_{\text{c}}, \epsilon, M) = (q_0, \epsilon)$

$\delta(q_0, \epsilon, \perp) = (\text{accept}, \perp)$

$\delta(q \text{ (aktueller Zustand)}, a \text{ (Eingabesymbol)}, X \text{ (oberstes Symbol auf dem Stack)}) = (q' \text{ (neuer Zustand)}, y \text{ (push)})$

Übergangsfunktionen als Tabelle (übersichtlicher)

Zustand	Eingabe	oberstes Symbol auf dem Stack	neuer Zustand	push
q0	a	⊥	q0	M ⊥
q0	b	⊥	q0	⊥
q0	c	⊥	q0	P P ⊥
q0	a	M	q0	M M
q0	b	M	q0	M
q0	c	M	q_c	ε
q0	a	P	q0	ε
q0	b	P	q0	P
q0	c	P	q0	P P P
q_c	ε	⊥	q0	P ⊥
q_c	ε	M	q0	ε
q0	ε	⊥	accept	⊥

Strategie:

- zwei Plus für jedes C
- ein Minus für jedes A
- Wenn der Stack leer ist, M für minus einsetzen und diese immer zuerst abbauen, bevor man P's hinzufügt
- ein Wort darf auch nur aus b oder mehreren b's bestehen, da $2*c = a$ und $2*0 = 0$
- nicht deterministisch möglich, wegen des letzten Übergangs zu dem Zustand "accept"

2. Beschreiben Sie Schritt für Schritt, wie der PDA die Eingaben bcaba und bccac abarbeitet.

bcaba

Zustand	Eingabe	oberstes Symbol auf dem Stack	neuer Zustand	push
q0	b	⊥	q0	⊥
q0	c	⊥	q0	P P ⊥

q0	a	P	q0	ϵ
q0	b	P	q0	P
q0	a	P	q0	ϵ
q0	ϵ	$\perp$	accept	$\perp$

- Eingabe gültig

bccac

Zustand	Eingabe	oberstes Symbol auf dem Stack	neuer Zustand	push
q0	b	$\perp$	q0	$\perp$
q0	c	$\perp$	q0	P P $\perp$
q0	c	P	q0	P P P
q0	a	P	q0	ϵ
q0	c	P	q0	P P P

- Stack ist nicht leer (Stack: PPPPPZ), daher kann Zustand q0 nicht zu accept übergehen
-> Eingabe ungültig

Nr. 2

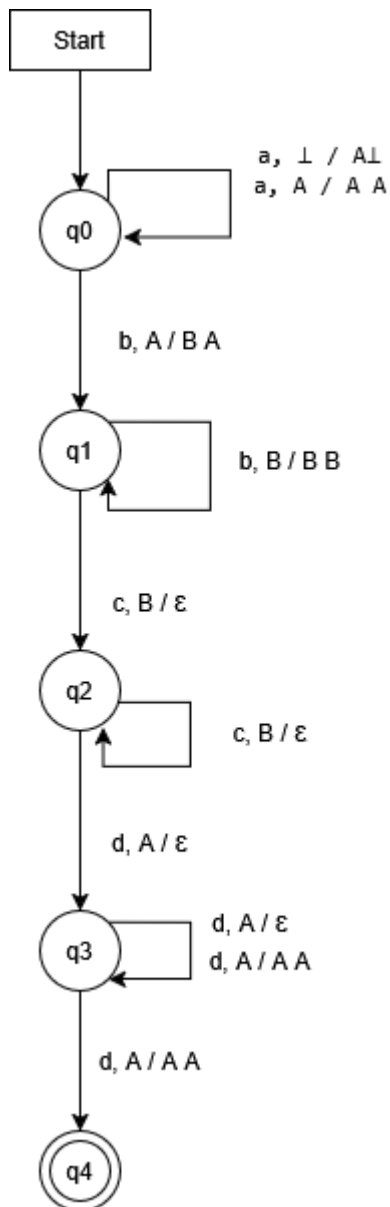
folgender PDA:

Zustand	Eingabe	oberstes Symbol auf dem Stack	neuer Zustand	push
q0	a	$\perp$	q0	A $\perp$
q0	a	A	q0	A A
q0	b	A	q1	B A
q1	b	B	q1	B B
q1	c	B	q2	ϵ
q2	c	B	q2	ϵ
q2	d	A	q3	ϵ
q3	d	A	q3	ϵ
q3	d	A	q3	A A
q4	ϵ	$\perp$	q4	ϵ

1. Ist der folgenden PDA deterministisch? Warum (nicht)?

- Der PDA ist nicht deterministisch, da von Zustand q_3 und der Eingabe d oder ϵ , bei dem obersten Symbol von A , entweder kein push stattfindet oder A zwei Mal gepusht wird. Für eine Kombination aus einem Zustand, einer Eingabe und einem Stacktop darf es nicht mehrere sonst gleiche Regeln geben, die zu zwei unterschiedlichen Stack-pushverhalten führen.

2. Zeichnen Sie den Automaten.



3. Geben Sie das 7-Tupel des PDA an.

(Q (Zustände) , Σ (Eingabesymbole) , Γ (Stackalphabet) , δ (Übergangsfunktionen), q_0 (Startzustand) , $\perp$ (Leerer Stack) , F (akzeptierende Zustände/Endzustände))

PDA = ($Q = \{q_0, q_1, q_2, q_3, q_4\}$, $\Sigma = \{a, b, c, d\}$, $\Gamma = \{A, B, \perp\}$, δ (Übergangsfunktionen), $q_0, \perp$, $F = \{q_4\}$)

4. Welche Sprache akzeptiert er?

Der PDA akzeptiert eine Sprache mit:

- 1 oder mehr a's am Anfang
- danach 1 oder mehr b's
- danach genau so viele c's wie b's
- zum Schluss mindestens so viele d's wie es a's am Anfang gab.

Nr.3

$G = (\{ \text{Statement} , \text{Condition} , \dots \} , \{ \text{"if"} , \text{"else"} , \dots \} , P , \text{Statement})$

$P = \{$

$\text{Statement} \rightarrow \text{"if"} \text{ Condition Statement} \mid \text{"if"} \text{ Condition Statement "else"} \text{ Statement}$

$\text{Condition} \rightarrow \dots$

$\}$

Die Sprache die die Grammatik beschreibt, lässt vieles offen beinhaltet aber, dass

Nicht terminale: Statement, Condition, etc.

Terminale: if, else, etc

doStuff wäre ein terminal mit

$\text{Statement} \rightarrow \text{doStuff} \mid \text{doStuff Statement}$

Bei dem folgenden input gäbe es 2 valide Bäume

if c doStuff if c doStuff else doStuff

kann

$\text{if c doStuff} \rightarrow (\text{if c doStuff}) \text{ else doStuff}$

und auch

$\text{if c doStuff} \rightarrow (\text{if c doStuff else doStuff})$

sein

-> demnach ist die Grammatik Mehrdeutig

Nr.4

Die entwickelte Grammatik ist:

$S \rightarrow S1 \mid S2$

$S1 \rightarrow AB$

$A \rightarrow aAb \mid \epsilon$

$B \rightarrow Bc \mid \epsilon$

$S2 \rightarrow CD$

$C \rightarrow aC \mid \epsilon$

$D \rightarrow bDc \mid \epsilon$

sie ist mehrdeutig da wir das selbe wort (zb "aabbcc") auf mehrere Arten erzeugen können

S

S1

AB

aAbB

aaAbbB

aabbB

aabbBc

aabbBcc

aabbcc

oder

S

S2

CD

aCD

aaCD

aaD

aabDc

aabbDcc

aabbcc

Nicht-deterministischer Sprung zu Branch A oder B

$$\delta(q_0, \varepsilon, \perp) = \{(q_A, \perp), (q_B, \perp)\}$$

BRANCH A: Zähle a's, ziehe b's ab, dann beliebig viele c's

$$\delta(q_A, a, \perp) = \{(q_A, \text{🚀} \perp)\}$$

$$\delta(q_A, a, \text{🚀}) = \{(q_A, \text{🚀} \text{🚀})\}$$

$$\delta(q_A, b, \text{🚀}) = \{(q_A, \varepsilon)\}$$

$$\delta(q_A, \varepsilon, \perp) = \{(q_{A_end}, \perp)\}$$

$$\delta(q_{A_end}, c, \perp) = \{(q_{A_end}, \perp)\}$$

BRANCH B: Beliebige viele a's, zähle b's, ziehe c's ab

$$\delta(q_B, a, \perp) = \{(q_B, \perp)\}$$

$$\delta(q_B, b, \perp) = \{(q_B, \text{🦄} \perp)\}$$

$$\delta(q_B, b, \text{🦄}) = \{(q_B, \text{🦄} \text{🦄})\}$$

$$\delta(q_B, c, \text{🦄}) = \{(q_B, \varepsilon)\}$$

$$\delta(q_B, \varepsilon, \perp) = \{(q_{B_end}, \perp)\}$$